

Basic C Without Tears

Guía de Referencia: El Arte del Pseudocódigo en C

Conceptos, Estándares de Escritura y Equivalencias con Código Compilable

Autor: Stack

1. ¿Qué es el Pseudocódigo y cuándo utilizarlo?

El **pseudocódigo** es una descripción estructurada y rigurosa de un algoritmo. No depende de la sintaxis estricta de un compilador (puntos y coma, cabeceras o tipos de formateo), pero **no es lenguaje coloquial informal**.

- **Diseño previo:** Permite estructurar la solución antes de lidiar con la memoria y punteros en C.
- **Evaluaciones formativas y entrevistas:** Permite evaluar el razonamiento algorítmico y el control de flujo sin penalizar por detalles sintácticos menores de bibliotecas externas.
- **Regla de Oro:** Todo bloque debe tener delimitadores explícitos (SI... FIN_SI, MIENTRAS... FIN_MIENTRAS) y respetar sangría visual (*indentación*).

2. Tabla de Equivalencias Rápidas: Sintaxis C vs. Pseudocódigo Estándar

Operación	Código Compilable en C	Pseudocódigo Válido
Asignación de variable	<code>total = 0;</code>	<code>total ← 0 ó total = 0</code>
Lectura de datos	<code>scanf("%d", &n);</code>	<code>LEER n</code>
Impresión en consola	<code>printf("Res: %d\n", x);</code>	<code>ESCRIBIR "Res: ", x</code>
Condicional simple	<code>if (cond) { ... }</code>	<code>SI cond ENTONCES ... FIN_SI</code>
Condicional doble	<code>if (c) { ... } else { ... }</code>	<code>SI c ENTONCES ... SINO ... FIN_SI</code>
Bucle con contador	<code>for (int i=0; i<n; i++)</code>	<code>PARA i ← 0 HASTA n-1 HACER</code>
Bucle condicional previo	<code>while (cond) { ... }</code>	<code>MIENTRAS cond HACER ... FIN_MIENTRAS</code>
Bucle con post-condición	<code>do { ... } while (cond);</code>	<code>REPETIR ... HASTA QUE cond</code>
Acceso a arreglos	<code>A[i] = A[i - 1];</code>	<code>A[i] ← A[i - 1]</code>
Retorno de valor	<code>return resultado;</code>	<code>RETORNAR resultado</code>

3. Comparativas Prácticas: Código C vs. Pseudocódigo

Caso 1: Validación y Clasificación de Datos (Control Condicional)

Problema: Recibir tres medidas flotantes, verificar si conforman un triángulo admisible y clasificarlo por sus lados.

Código Compilable en C (C99/C11)

```
1 #include <stdio.h>
2
3 void clasificar_triangulo(float a, float b, float c) {
4     // 1. Validacion de desigualdad triangular
5     if (a < b + c && b < a + c && c < a + b) {
6         printf("Triangulo valido: ");
7         // 2. Clasificacion por igualdad de lados
8         if (a == b && b == c) {
9             printf("Equilatero\n");
10        } else if (a == b || b == c || a == c) {
11            printf("Isosceles\n");
12        } else {
13            printf("Escaleno\n");
14        }
15    } else {
16        printf("No conforma un triangulo.\n");
17    }
18 }
```

Pseudocódigo Válido y Riguroso

```
1 FUNCION clasificar_triangulo(a, b, c)
2     // 1. Validar la desigualdad triangular
3     SI (a < b + c) Y (b < a + c) Y (c < a + b) ENTONCES
4         ESCRIBIR "Triangulo valido: "
5
6     // 2. Clasificacion por igualdad de lados
7     SI (a = b) Y (b = c) ENTONCES
8         ESCRIBIR "Equilatero"
9     SINO SI (a = b) O (b = c) O (a = c) ENTONCES
10        ESCRIBIR "Isosceles"
11    SINO
12        ESCRIBIR "Escaleno"
13    FIN_SI
14    SINO
15        ESCRIBIR "No conforma un triangulo."
16    FIN_SI
17 FIN_FUNCION
```

Clave Didáctica del Caso 1

En el pseudocódigo nos enfocamos en la lógica booleana (Y, O) y el anidamiento limpio. No es necesario gestionar bibliotecas (`#include <stdio.h>`), prototipos ni secuencias de escape (`\n`).

Caso 2: Búsqueda y Filtrado en Arreglos Unidimensionales

Problema: Recorrer un arreglo A de n elementos y almacenar en picos aquellos valores estrictamente mayores a sus dos vecinos contiguos ($1 \leq i \leq n - 2$).

Código Compilable en C (C99/C11)

```
1 #include <stdio.h>
2
3 int filtrar_picos(const int A[], int n, int picos[]) {
4     int total_picos = 0;
5
6     // Recorrido seguro de los índices interiores
7     for (int i = 1; i < n - 1; i++) {
8         if (A[i] > A[i - 1] && A[i] > A[i + 1]) {
9             picos[total_picos] = A[i];
10            total_picos++;
11        }
12    }
13    return total_picos; // Retorna cantidad detectada
14 }
```

Pseudocódigo Válido y Riguroso

```
1 FUNCION filtrar_picos(A, n, picos)
2     total_picos <- 0
3
4     // Iterar estrictamente sobre elementos interiores
5     PARA i <- 1 HASTA n - 2 HACER
6         SI (A[i] > A[i - 1]) Y (A[i] > A[i + 1]) ENTONCES
7             picos[total_picos] <- A[i]
8             total_picos <- total_picos + 1
9         FIN_SI
10    FIN_PARA
11
12    RETORNAR total_picos
13 FIN_FUNCION
```

Clave Didáctica del Caso 2

Nota cómo el pseudocódigo respeta con precisión matemática los **límites del arreglo**: el ciclo recorre desde 1 hasta $n - 2$. Esta exactitud en los índices distingue a un pseudocódigo universitario sólido de una descripción informal vaga.

Caso 3: Cadenas de Caracteres y Detección de Centinela ('\0')

Problema: Determinar la longitud de una cadena recorriendo carácter por carácter hasta hallar el terminador nulo.

Código Compilable en C (C99/C11)

```

1 int longitud_cadena(const char str[]) {
2     int len = 0;
3     while (str[len] != '\0') {
4         len++;
5     }
6     return len;
7 }

```

Pseudocódigo Válido y Riguroso

```

1 FUNCION longitud_cadena(str)
2     len <- 0
3     MIENTRAS str[len] != '\0' HACER
4         len <- len + 1
5     FIN_MIENTRAS
6     RETORNAR len
7 FIN_FUNCION

```

4. Criterios de Validez: Lo que SÍ se permite vs. Lo que INVALIDA una respuesta**[VÁLIDO] Prácticas Permitidas**

Omitir directivas de compilación (`#include`) y tipos de formato (`%d,%f`).

Usar operadores intuitivos: `<-` o `=` para asignar, y palabras en español como Y, O, NO.

Omitir llaves `{}` siempre que la indentación y palabras de cierre delimiten el bloque.

Indexar arreglos y cadenas de texto mediante corchetes familiares `A[i]` o `str[k]`.

[INVÁLIDO] Errores Graves

Escribir frases ambiguas: «*Buscar el mayor elemento*» sin escribir el bucle de búsqueda.

Inventar funciones mágicas que resuelvan el núcleo del problema algorítmico pedido.

Dejar bucles sin condición explícita de parada o variables acumuladoras sin inicializar.

Omitir los cierres de estructuras (`FIN_SI`, `FIN_MIENTRAS`), volviendo ambiguo el anidamiento.